

Wstęp do Javy

Od podstaw do pierwszych programów

Typy danych • Zmienne • Pętle • Metody • OOP • Kolekcje

Typy danych prymitywnych

int

32-bit

Liczby całkowite

```
int x = 42;
```

double

64-bit

Liczby zmiennoprzecinkowe

```
double pi = 3.14;
```

boolean

1-bit

Wartość logiczna

```
boolean ok = true;
```

char

16-bit

Pojedynczy znak Unicode

```
char c = 'A';
```

long

64-bit

Duże liczby całkowite

```
long big = 999L;
```

float

32-bit

Zmiennoprzecinkowe 32b

```
float f = 3.14f;
```

byte

8-bit

Liczby całkowite -128..127

```
byte b = 127;
```

short

16-bit

Liczby całkowite ± 32767

```
short s = 1000;
```

Zmienne, stałe i typ String

```
// Deklaracja i inicjalizacja zmiennych
int wiek = 20;
double pensja = 5000.50;
String imie = "Anna";
boolean czyStudent = true;

// Stała - nie można zmienić wartości
final double PI = 3.14159;

// Var - inferencja typów (Java 10+)
var liczba = 42;    // kompilator sam rozpozna int
```



Konwencja nazw

camelCase: mojaZmienna
Klasy: MojaKlasa
Stałe: MOJA_STALA



final

Słowo kluczowe final tworzy stałą – nie można jej ponownie przypisać wartości.



String

String nie jest prymitywem!
To klasa. Używamy " " do tworzenia tekstu.



Rzutowanie typów:
`int x = (int) 3.99; // → 3`

Pobieranie danych – klasa Scanner

```
import java.util.Scanner;

public class PobieranieDanych {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Podaj imię: ");
        String imie = sc.nextLine();

        System.out.print("Podaj wiek: ");
        int wiek = sc.nextInt();

        System.out.println("Cześć " + imie + ", masz " +
            wiek + " lat!");
        sc.close();
    }
}
```

nextLine()

Odczytuje całą linię
jako String

nextInt()

Odczytuje
liczbę całkowitą

nextDouble()

Odczytuje liczbę
zmiennoprzecinkową

next()

Odczytuje jedno
słowo (bez spacji)

sc.close()

Zamknij Scanner
po użyciu!

Operatory

Arytmetyczne

- `+` Dodawanie
- `-` Odejmowanie
- `*` Mnożenie
- `/` Dzielenie
- `%` Reszta z dzielenia

Porównania

- `==` Równy
- `!=` Różny
- `>` Większy
- `<` Mniejszy
- `>=` Większy lub równy

Logiczne

- `&&` AND (i)
- `||` OR (lub)
- `!` NOT (nie)

Przypisania

- `=` Przypisanie
- `+=` Dodaj i przypisz
- `-=` Odejmij i przypisz
- `++` Inkrementacja
- `--` Dekrementacja

Instrukcje warunkowe – if / else / switch

```
// if - else if - else
int ocena = 85;
if (ocena >= 90) {
    System.out.println("Bardzo dobry");
} else if (ocena >= 75) {
    System.out.println("Dobry");
} else {
    System.out.println("Dostateczny");
}
```

```
// switch
String dzien = "PON";
switch (dzien) {
    case "PON" ->
        System.out.println("Poniedziałek");
    case "WT" ->
        System.out.println("Wtorek");
}
```

Operator Ternary

```
// warunek ? jeśliTrue : jeśliFalse
int a = 10, b = 20;
int max = (a > b) ? a : b;
// max = 20
```

Zasady stosowania if:

- Nawiasy klamrowe {} zawsze!
- switch → dla wielu wartości jednej zmiennej
- Ternary → krótkie, proste warunki
- Unikaj głębokiego zagnieżdżenia

Pętle – for, while, do-while

Pętla for

```
// Znana liczba iteracji
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

// for-each (po kolekcji)
for (String s : lista) {
    System.out.println(s);
}
```

Pętla while

```
// Gdy nie znamy liczby iteracji
int n = 1;
while (n <= 10) {
    System.out.println(n);
    n++;
}
```

Pętla do-while

```
// Wykonuje się co najmniej raz
int n = 1;
do {
    System.out.println(n);
    n++;
} while (n <= 5);
```

